

Network Yield and Computing Allocation: A Unified Framework for Distributed Task Scheduling via Thermodynamic Resolution Floors, Cell-Partition Control, and Market Clearing

Kundai Farai Sachikonye

July 21, 2026

Abstract

We develop a self-contained theory of distributed computing allocation in which every schedulable resource, every state observation, and every economic signal shares a single irreducible quantum $\tau_0 > 0$: the *compute-tick*. The framework rests on three foundations assembled into a unified object. First, a *finite observer axiom* forces every monitor to return a cell rather than a point, establishing a resolution floor $\beta \geq \tau_0 > 0$ that bounds observation precision absolutely. Second, a *cell-partition scheduler* compiled from offline stability analysis governs task placement with a Lyapunov guarantee and a Banach contraction whose ratio is computable without runtime measurement. Third, a *separation-cost functional* assigns to every execution slot a yield price equal to its marginal contribution to useful task progress, and the clearing map for those prices coincides with the yield-optimal assignment and with deterministic scheduler closure at the same fixed point.

The central result is a *Three-way Equivalence*: yield-optimality, deterministic closure, and market clearing are one fixed point, coinciding because τ_0 serves simultaneously as the physical resolution floor for observations, the algorithmic closure threshold for the scheduler, and the minimum lot size for the clearing market. From this equivalence we derive, as theorems rather than assumptions, that executor threads run at forced slot-optimal utilisation ratios, that tasks sort onto execution slots by comparative advantage in compute-ticks, and that a purely local, residual-work-descent routing law drains every task queue in finite time

because the separation cost raised by a stalled queue is identical to the yield signal that summons an executor. A structural incorruptibility theorem guarantees that no monitoring payload can influence the scheduling action stream without traversing an explicit approval boundary. Finally, we invert the locus of agency: an allocated process is re-read as a *persistent goal-directed agent* that pursues its own target every tick, succeeds to a fresh goal on completion rather than terminating, returns a distinct response on each interaction because its inner state has advanced, and summons capacity by raising its own price when stalled—each of these properties a theorem of the same machinery, with goal succession the one added primitive. All proofs are given in full; no prior companion paper is required.

Contents

1	Introduction	2
2	The Computing Network Model	3
2.1	Primitive objects	3
2.2	Separation cost	3
2.3	Assignments and closure	3
3	The Resolution Floor	4
3.1	Finite observer axiom	4
4	Monitoring Architecture	4
4.1	S-entropy state space	4
4.2	Cell partition and the monitoring cell	4
4.3	Trajectory completion	5
4.4	Closure dichotomy for monitoring	5

4.5	Coherence and sensor federation . . .	5
4.6	Monitor–control separation theorem . . .	6
5	Cell-Partition Scheduler	6
5.1	Offline compilation	6
5.2	Stability	6
5.3	Discrete-time contraction	7
5.4	CUSUM–cell-crossing identity . . .	7
6	Separation Costs and Network Yield	8
6.1	Slot-level separation costs	8
6.2	The yield market	8
7	The Three-Way Equivalence	8
7.1	Forced optimal utilisation	9
7.2	Comparative-advantage sorting . . .	10
8	Liveness Under Backpressure	10
8.1	Backpressure routing	10
8.2	Liveness theorem	10
9	Structural Incorruptibility	10
10	Swarm Coordination and Phase-Lock	11
11	Processes as Persistent Agents	11
11.1	The process agent	12
11.2	Occupied, not waiting	12
11.3	Goal succession: persistence past completion	12
11.4	An ever-fresh response	13
11.5	The agent initiates	13
11.6	Persistence is incorruptible	14
12	Numerical Validation	14
12.1	Formula checks	14
12.2	A corrected theorem	14
12.3	Agent-network simulation	14
13	Discussion	15
13.1	Relation to backpressure scheduling . . .	15
13.2	Relation to fair queueing	15
13.3	Relation to cluster scheduling . . .	15
13.4	The resolution floor as a thermodynamic constraint	15
13.5	The compute-tick as the unifying quantum	15
13.6	Monitor–control separation as a design principle	15
13.7	Processes as agents	16
13.8	Limitations	16
14	Conclusion	17

1 Introduction

Distributed computing systems schedule tasks onto processors, manage queues of pending work, observe system state, and route tasks between machines. Despite decades of theory—backpressure scheduling [10], fluid models [5], fair queueing [2], cluster managers [3, 11]—no unified account explains simultaneously why a correct scheduler *cannot* observe state with arbitrary precision, why the scheduler that maximises useful throughput is the same one that clears a natural market, and why both coincide with the fixed point of a pure local rule requiring no global coordination.

This paper provides that account. The central object is a *computing network* $\mathcal{G} = (\mathcal{N}, \mathcal{E})$: a finite set of *queue nodes* $\mathcal{N}$ (buffers for pending tasks) and a finite set of *execution slots* $\mathcal{E}$ (processors), each with a capacity and a cost. Tasks carry private completion targets; the objective is useful target-ward progress per unit compute-tick. The key insight is that a single positive quantum $\tau_0 > 0$ —the minimum schedulable unit of processor time—appears in three distinct roles:

- *Physical*: the resolution floor below which no monitor can distinguish system states;
- *Algorithmic*: the gap below which the scheduler cannot improve by reassigning a single task;
- *Economic*: the minimum lot size in the clearing market for execution capacity.

When these three roles are played by the same τ_0 , the three fixed-point conditions coincide and the Three-way Equivalence holds. When the roles are played by distinct constants, the equivalence degrades gracefully: the slack grows to the maximum of the three constants and the clearing price deviates from the separation cost by the algorithmic-economic split.

Organisation. Section 2 defines the computing network model and all primitive objects. Section 3 establishes the resolution floor theorem. Section 4 develops the monitoring architecture: S-entropy state space, trajectory completion operator, and the monitor–control separation theorem. Section 5 presents the cell-partition scheduler with its stability and contraction theorems. Section 6 defines separation costs and the yield functional. Section 7 proves the Three-way Equivalence. Section 8 proves liveness under backpressure coupling. Section 9 proves structural incorruptibility.

Section 11 inverts the locus of agency, recasting allocated processes as persistent goal-directed agents and adding the goal-succession primitive. Section 13 situates the results in the literature. Section 14 concludes.

Prerequisites. The paper is self-contained. Familiarity with basic real analysis (metric spaces, fixed-point theorems) and linear algebra (eigenvalues, positive-definite matrices) suffices.

2 The Computing Network Model

2.1 Primitive objects

Definition 2.1 (Computing network). A *computing network* is a tuple $\mathcal{G} = (\mathcal{N}, \mathcal{E}, c, \ell, \bar{v})$ where:

- $\mathcal{N} = \{n_1, \dots, n_K\}$ is a finite set of *queue nodes* (buffers);
- $\mathcal{E} = \{e_1, \dots, e_M\}$ is a finite set of *execution slots* (processors);
- $c : \mathcal{E} \rightarrow \mathbb{Z}_{>0}$ assigns each slot a *thread capacity* $c(e) \geq 1$;
- $\ell : \mathcal{E} \rightarrow \mathbb{R}_{>0}$ assigns each slot an *expected task duration* $\ell(e) > 0$;
- $\bar{v} : \mathcal{E} \rightarrow \mathbb{R}_{>0}$ assigns each slot a *maximum throughput rate* $\bar{v}(e) > 0$.

Definition 2.2 (Compute-tick). The *compute-tick* $\tau_0 > 0$ is the minimum indivisible unit of processor time. No scheduling decision, observation, or price can be expressed to a precision finer than τ_0 .

Remark 2.3. In practice τ_0 corresponds to the OS scheduler quantum, the minimum time-slice allocated by the kernel. Its existence is not an engineering choice but a consequence of the finite-observer axiom proved in Section 3.

Definition 2.4 (Task). A *task* x is a tuple $(o(x), \tau(x), w(x))$ where $o(x) \in \mathcal{N}$ is the *originating queue node*, $\tau(x) \in \mathbb{R}^d$ is the *completion target* (a feature vector encoding the desired output state), and $w(x) > 0$ is the *work estimate* (expected compute-ticks to completion).

Definition 2.5 (Residual work). For a task x currently executing on slot e , the *residual work* is

$$r(x, e) = \rho(\text{loc}(x), \tau(x)),$$

where $\text{loc}(x) \in \mathbb{R}^d$ is the current state of x 's computation (the partial result produced so far) and ρ is the Euclidean distance on $\mathbb{R}^d$.

Definition 2.6 (Queue occupancy and directional pressure). Let $\mathcal{X}(n)$ denote the set of tasks currently queued at node $n \in \mathcal{N}$, and let $\text{hd}(e) \in \mathbb{R}^d$ denote the “head” of slot e : the state at which a task assigned to e arrives after one tick of execution. The *directional pressure* from node n towards slot e is

$$P(n, e) = \sum_{x \in \mathcal{X}(n)} [r(x, e) - \rho(\text{hd}(e), \tau(x))]^+,$$

where $[z]^+ = \max(z, 0)$. Tasks that would *not* make progress by moving to e contribute zero pressure.

Definition 2.7 (Network transport yield). The *network transport yield* of an assignment $A : \mathcal{X} \rightarrow \mathcal{E}$ (a map from tasks to slots) is

$$\text{yield}(A) = \frac{\sum_{x \in \mathcal{X}} [r(x, A(x)) - \rho(\text{hd}(A(x)), \tau(x))]^+}{\sum_{e \in \mathcal{E}} \tau_0 \cdot c(e) \cdot g_u(v(e))},$$

where $v(e) \in (0, \bar{v}(e)]$ is the throughput rate at which slot e operates under assignment A , and $g_u : \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0}$ is a strictly convex *utilisation cost* satisfying $g'_u > 0$ and $g''_u > 0$.

The numerator counts total target-ward progress; the denominator counts total resource consumption in compute-ticks weighted by utilisation cost.

2.2 Separation cost

Definition 2.8 (Separation cost). The *separation cost* of slot e under assignment A is the marginal yield lost by removing e from the network:

$$\text{sep}(e, A) = \text{yield}(A) - \text{yield}(A|_{\mathcal{E} \setminus \{e\}}),$$

where $A|_{\mathcal{E} \setminus \{e\}}$ is the optimal re-assignment of tasks after removing e .

Remark 2.9. $\text{sep}(e, A) \geq 0$. A redundant slot (e has a perfect substitute) has $\text{sep} = 0$; a bottleneck slot has large sep .

2.3 Assignments and closure

Definition 2.10 (Single-slot reassignment neighbourhood). Two assignments A and A' are τ_0 -*adjacent* if they differ in the allocation of at most one task and the resulting change in yield is at most τ_0 .

Definition 2.11 (Deterministic closure). Assignment A is *deterministically closed* if no τ_0 -adjacent assignment A' satisfies $\text{yield}(A') > \text{yield}(A) + \tau_0$. Equivalently, the best single-task reassignment cannot improve yield beyond the resolution floor.

3 The Resolution Floor

3.1 Finite observer axiom

Assumption 3.1 (Finite observer). Any monitoring subsystem of the computing network is itself a finite computational process: it runs on hardware with a finite number of distinguishable states, communicates over channels with finite bandwidth, and terminates its observation procedure in finite time.

This axiom is not a limitation of current technology; it is a logical consequence of a bounded physical substrate. It implies the following foundational theorem.

Theorem 3.2 (Resolution floor). *Under Assumption 3.1, there exists $\beta > 0$ such that for every observable quantity q and every monitor $\mathcal{M}$, the identification $\mathcal{M}(q)$ satisfies*

$$\inf_q \text{diam}(\mathcal{M}(q)) \geq \beta > 0,$$

where $\text{diam}(\cdot)$ is the diameter of the set $\mathcal{M}(q) \subseteq \mathbb{R}$ returned by $\mathcal{M}$.

No monitor returns a point; every identification carries strictly positive irreducible residual uncertainty.

Proof. Let S be the finite state space of $\mathcal{M}$ and let Q be the range of q . $\mathcal{M}$ defines a function $f : Q \rightarrow 2^S$. Since S is finite, f has at most $|S|$ distinct outputs. The pre-images $\{f^{-1}(r) : r \in \text{range}(f)\}$ partition Q into at most $|S|$ cells. Since Q is a metric space with at least $|S| + 1$ distinct points (for any finite S , the range of a physical quantity is infinite in principle), at least one cell has diameter $\geq \text{diam}(Q)/|S| > 0$. Setting $\beta = \min_r \text{diam}(f^{-1}(r))$ and noting that every cell must have positive diameter (because q is a continuous physical quantity) gives $\beta > 0$.

Formally: suppose for contradiction that some cell $f^{-1}(r)$ has diameter zero. Then $f^{-1}(r)$ is a single point, and f distinguishes that point from all others—which requires $|S| \geq |Q|$. But Q is uncountable (it is a real-valued quantity) and S is finite, a contradiction. $\square$

Corollary 3.3 (Compute-tick as floor). *The compute-tick τ_0 satisfies $\tau_0 \geq \beta$. Scheduling decisions made at resolution τ_0 are consistent with the observation resolution guaranteed by Theorem 3.2.*

Proof. Any scheduling decision that distinguishes two system states separated by less than β would require a monitor that violates Theorem 3.2. Therefore, the minimum indivisible decision granularity must be $\geq \beta$. Setting $\tau_0 = \beta$ achieves the tightest consistent granularity. $\square$

4 Monitoring Architecture

4.1 S-entropy state space

Definition 4.1 (S-normalisation). For a physical quantity q with known bounds $[q_{\min}, q_{\max}]$, the *S-normalisation* is

$$S(q) = \frac{q - q_{\min}}{q_{\max} - q_{\min}} \in [0, 1].$$

S-normalisation maps heterogeneous dimensional quantities onto a common unit interval, enabling uniform treatment of queue lengths, utilisation rates, task progress fractions, and thermal metrics.

Definition 4.2 (S-entropy state space). Let the observable state of the computing network be described by m independent quantities $q_1, \dots, q_m$. The *S-entropy state space* is

$$\mathcal{H} = [0, 1]^m,$$

equipped with the Euclidean metric. Each coordinate $S(q_i)$ represents one observable dimension. $\mathcal{H}$ is compact, connected, and a complete metric space.

Remark 4.3. The three canonical coordinates are: $s_k = S(\text{queue occupancy})$ (knowledge entropy), $s_t = S(\text{task age})$ (temporal entropy), and $s_e = S(\text{residual work fraction})$ (evolution entropy). Additional coordinates may encode thermal load, network latency, and memory pressure.

4.2 Cell partition and the monitoring cell

Definition 4.4 (Cell partition). A *cell partition* $\mathcal{C} = \{C_1, \dots, C_N\}$ is a finite Borel partition of $\mathcal{H}$ such that each cell C_i has width $w_i \geq \beta > 0$.

Definition 4.5 (Monitoring cell). For an observed state $h \in \mathcal{H}$, the *monitoring cell* is the unique $C_i \in \mathcal{C}$ containing h :

$$\text{cell}(h) = C_i \iff h \in C_i.$$

The monitor returns $\text{cell}(h)$, not h itself. This is consistent with Theorem 3.2: the cell has diameter $\geq \beta > 0$.

4.3 Trajectory completion

Computing networks exhibit partially observable dynamics: a task’s computational trajectory from initiation to completion is not directly accessible but must be inferred from snapshots of residual work.

Definition 4.6 (Fuzzy network state). A *fuzzy network state* is a function $\mu : \mathcal{N} \times \mathcal{E} \rightarrow [0, 1]$ assigning to each (queue-node, slot) pair a belief weight, with $\sum_{n,e} \mu(n, e) = 1$.

Definition 4.7 (State space of fuzzy states). Let $\mathcal{F}$ be the set of all fuzzy network states. Equipped with the ℓ^1 metric $d_1(\mu, \nu) = \sum_{n,e} |\mu(n, e) - \nu(n, e)|$, $(\mathcal{F}, d_1)$ is a complete metric space.

Definition 4.8 (Trajectory completion operator). Define the *trajectory completion operator* $\mathcal{T} = \mathcal{T}_{\text{kin}} \circ \mathcal{T}_{\text{therm}} \circ \mathcal{T}_{\text{bal}}$ on $\mathcal{F}$:

- $\mathcal{T}_{\text{bal}}$ rebalances belief weights to be consistent with observed queue occupancies;
- $\mathcal{T}_{\text{therm}}$ applies a diffusion step consistent with the maximum-entropy principle over unobserved paths;
- $\mathcal{T}_{\text{kin}}$ projects forward by one tick using the assignment kinematics.

Theorem 4.9 (Trajectory completion converges). *Under Assumption 3.1 and with a fixed assignment A , the operator $\mathcal{T}$ is a Banach contraction on $(\mathcal{F}, d_1)$ with contraction ratio $\rho_{\mathcal{T}} < 1$. In particular, there exists a unique fixed point $\mu^* \in \mathcal{F}$ with $\mathcal{T}(\mu^*) = \mu^*$, and for any initial $\mu_0 \in \mathcal{F}$,*

$$d_1(\mathcal{T}^k(\mu_0), \mu^*) \leq \rho_{\mathcal{T}}^k \cdot d_1(\mu_0, \mu^*),$$

so the sequence $(\mathcal{T}^k(\mu_0))$ converges geometrically to μ^ .*

Proof. $\mathcal{T}_{\text{bal}}$ is a projection onto the closed convex set of distributions consistent with observations; as a nearest-point projection in ℓ^1 onto a convex set, it is non-expansive. $\mathcal{T}_{\text{therm}}$ applies a strictly positive stochastic matrix Q

(the maximum-entropy diffusion kernel); by the Perron–Frobenius theorem, Q has a unique stationary distribution and its spectral gap is $1 - \lambda_2(Q) > 0$, so multiplication by Q is a contraction in ℓ^1 with ratio $\leq 1 - \text{gap}(Q) < 1$. $\mathcal{T}_{\text{kin}}$ is a deterministic permutation of the state; it is an isometry, hence non-expansive. The composition of a strict contraction with non-expansive maps is a strict contraction [1]. Setting $\rho_{\mathcal{T}} = 1 - \text{gap}(Q) < 1$ completes the proof. $\square$

4.4 Closure dichotomy for monitoring

Definition 4.10 (Monitoring confirmation). A monitoring process over a finite set of sources $\Gamma = \{\gamma_1, \dots, \gamma_n\}$ terminates in one of two states: *confirmed* (sufficient evidence accumulated) or *declined* (insufficient evidence; action withheld).

Theorem 4.11 (Confirm-or-decline dichotomy). *Every monitoring process over finitely many sources with the closure criterion (Definition 2.11 adapted to monitoring: no additional evidence reduces cell diameter below β) terminates in confirmed or declined. There is no third outcome.*

Proof. The monitoring process accumulates evidence $e_1, e_2, \dots$ from n finite sources. By Assumption 3.1, each source is a finite-state machine; the Cartesian product of n finite-state machines is finite. The process therefore explores a finite directed graph of evidence states. Under the closure criterion, the process halts when no new transition reduces the diameter of the identified cell below β . Since the graph is finite, the process must reach a terminal state in finite steps. Terminal states partition into: states where the current cell meets the confirmation threshold (confirmed) and states where it does not (declined). There is no cycle that avoids both (the acyclicity follows from the requirement that each step must either reduce diameter or contribute new evidence; in a finite graph this eventually exhausts). $\square$

4.5 Coherence and sensor federation

Definition 4.12 (Mutually supporting cycle). A set of monitoring sources $\{\gamma_1, \dots, \gamma_n\}$ forms a *mutually supporting cycle* if for each i , source γ_i can confirm the output of $\gamma_{i-1 \bmod n}$ independently of all other sources.

Theorem 4.13 (Coherence requires three). *A monitoring system is resilient to the arbitrary*

failure of any single source if and only if the remaining sources form a mutually supporting cycle of length ≥ 3 .

Proof. Necessity. If all cycles in the dependency graph have length ≤ 2 , then every pair of sources supports only each other. Removing either element of such a pair destroys all support for the other, so the system is not resilient to single-source failure.

Sufficiency. Suppose there exists a cycle $\gamma_1, \dots, \gamma_n$ with $n \geq 3$. Remove source γ_j for any j . The remaining cycle $\gamma_1, \dots, \gamma_{j-1}, \gamma_{j+1}, \dots, \gamma_n$ still has length $n-1 \geq 2$. For $n \geq 3$, $n-1 \geq 2$ and the cycle is non-trivial. By the definition of mutual support, γ_{j-1} is supported by γ_{j-2} and γ_{j+1} is supported by γ_{j+2} ; the chain remains intact. Monitoring evidence can still propagate around the cycle, and the confirm-or-decline dichotomy (Theorem 4.11) is preserved. $\square$

Definition 4.14 (Multiplicative confirmation composition). For independent confirmation sources $\gamma_1, \dots, \gamma_n$ each with confirmation strength $\kappa_i \in [0, 1]$, the *composite confirmation strength* is

$$\kappa(\gamma_1 \circ \dots \circ \gamma_n) = 1 - \prod_{i=1}^n (1 - \kappa_i).$$

4.6 Monitor–control separation theorem

Theorem 4.15 (Monitor–control separation). *A subsystem that acts on its own observations cannot simultaneously serve as an unbiased monitor of the system it controls.*

Formally: let $\mathcal{M}$ be a monitor and $\mathcal{C}$ be a controller for the same system Σ . If $\mathcal{C}$ ’s actions are a function of $\mathcal{M}$ ’s outputs, then $\mathcal{M}$ ’s outputs are not independent of $\mathcal{C}$ ’s actions. In particular, any τ_0 -precise action by $\mathcal{C}$ induces a selection bias in $\mathcal{M}$ of at least β , violating the unbiasedness condition.

Proof. Let $h \in \mathcal{H}$ be the true system state. $\mathcal{M}$ returns $\text{cell}(h)$ (Def. 4.5). $\mathcal{C}$ reads $\text{cell}(h)$ and executes action $u = A(\text{cell}(h))$. The action u modifies Σ and therefore h : the post-action state is $h' = f(h, u)$ for some dynamics f . If $\mathcal{M}$ subsequently observes h' to inform the same control cycle, its next output $\text{cell}(h')$ depends on u , hence on $\text{cell}(h)$, hence on the previous output of $\mathcal{M}$. The observation is therefore conditioned on the action, not independent of it.

For unbiasedness to hold, the monitor’s output distribution must have conditional expectation $\mathbb{E}[\text{cell}(h)|u] = \mathbb{E}[\text{cell}(h)]$ for all u . But $u = A(\text{cell}(h))$ makes u a deterministic function of $\text{cell}(h)$, so $h' = f(h, A(\text{cell}(h)))$; the expectation $\mathbb{E}[\text{cell}(h')|\text{cell}(h)]$ is a non-trivial conditional distribution, not the marginal. The selection bias is the KL divergence between the conditional and marginal; since cell has diameter $\geq \beta > 0$ and A maps distinct cells to distinct actions (by the schedulability condition), this divergence is at least $\log(1 + \beta/w_{\min}) > 0$, where $w_{\min}$ is the minimum cell width.

Therefore, monitoring and control must be separate processes whose outputs are never in the same feedback loop within one sampling period. $\square$

5 Cell-Partition Scheduler

5.1 Offline compilation

The cell-partition scheduler pre-computes a lookup table $A : \mathcal{C} \rightarrow \mathcal{E}^{\mathcal{X}}$ mapping each monitoring cell to a task-to-slot assignment. This computation is performed offline; at runtime the scheduler reads the monitoring cell and performs a single table lookup.

Definition 5.1 (Cell-partition scheduler). The *cell-partition scheduler* is the function

$$u(t) = A(\text{cell}(S(h(t)))),$$

where $h(t)$ is the observed state at time t , S applies S-normalisation (Def. 4.1), cell returns the monitoring cell (Def. 4.5), and A is the pre-compiled assignment table.

Remark 5.2. Because the cell partition has width $\geq \beta \geq \tau_0$, the scheduler operates at resolution consistent with Corollary 3.3. It never attempts to distinguish states separated by less than τ_0 .

5.2 Stability

Assumption 5.3 (Per-cell dynamics). For each cell $C_i \in \mathcal{C}$, the closed-loop system under assignment $A(C_i)$ has linearised dynamics $\dot{h} = F_i(h)$ with a quadratic Lyapunov function $V_i(h) = h^T P_i h$ satisfying

$$F_i^T P_i + P_i F_i \preceq -\alpha_i P_i, \quad \alpha_i > 0.$$

Theorem 5.4 (Piecewise Lyapunov stability). *Under Assumption 5.3 and with the cell-partition*

scheduler (Def. 5.1), the network state $h(t)$ converges globally asymptotically to the target cell $C^* \ni h^*$ containing the desired operating point.

Proof. Define the composite Lyapunov function

$$V(h) = V_i(h) \quad \text{for } h \in C_i.$$

On each cell C_i , $\dot{V}_i = h^T(F_i^T P_i + P_i F_i)h \leq -\alpha_i h^T P_i h = -\alpha_i V_i(h)$. At cell boundaries, the assignment switches from $A(C_i)$ to $A(C_j)$; by the cell-width condition ($w_i \geq \beta$), the boundary crossing requires a state change of magnitude $\geq \beta/2$, which is detectable. Across boundaries, V may increase by at most $\Delta_{ij} = \sup_{h \in C_i \cap \partial C_j} |V_j(h) - V_i(h)|$. Under the Nyquist-type bound (Theorem 5.5), the number of crossings per unit time is bounded, so the average V decrease per crossing exceeds the boundary increment. By LaSalle's invariance principle applied piecewise, $h(t) \rightarrow C^*$ as $t \rightarrow \infty$. $\square$

Theorem 5.5 (Nyquist-type cell-width bound). *For the cell-partition scheduler to avoid aliasing (sampling faster than the cell permits), the cell width must satisfy*

$$\beta \leq w_{\text{cell}} \leq \frac{1}{2B_{\text{proc}}},$$

where B_{proc} is the bandwidth of the fastest monitored process (inverse of the minimum observable period).

Proof. The lower bound $w_{\text{cell}} \geq \beta$ is Corollary 3.3. The upper bound follows from the Nyquist–Shannon sampling theorem [8]: to faithfully represent a signal of bandwidth B_{proc} , the sampling interval must be $\leq 1/(2B_{\text{proc}})$. Since each cell corresponds to one sampling interval, $w_{\text{cell}} \leq 1/(2B_{\text{proc}})$. $\square$

5.3 Discrete-time contraction

In practice, the scheduler operates in discrete time with step $h = \tau_0$.

Theorem 5.6 (Banach contraction for discrete scheduler). *Let the discrete dynamics under $A(C_i)$ be $h_{t+1} = h_t + \tau_0 F_i(h_t)$. Define the discrete operator $\mathcal{T}_i : C_i \rightarrow C_i$ by $\mathcal{T}_i(h) = h + \tau_0 F_i(h)$. If $\tau_0 < 2\alpha_i/L_i^2$ where L_i is the Lipschitz constant of F_i on C_i , then $\mathcal{T}_i$ is a Banach contraction with ratio*

$$\rho_i = \sqrt{1 - 2\tau_0\alpha_i + \tau_0^2 L_i^2} < 1.$$

Proof. For $h, h' \in C_i$:

$$\begin{aligned} \|\mathcal{T}_i(h) - \mathcal{T}_i(h')\|^2 &= \|(h - h') + \tau_0(F_i(h) - F_i(h'))\|^2 \\ &= \|h - h'\|^2 + 2\tau_0\langle h - h', F_i(h) - F_i(h') \rangle \\ &\quad + \tau_0^2 \|F_i(h) - F_i(h')\|^2. \end{aligned}$$

By Assumption 5.3, $2\langle h - h', F_i(h) - F_i(h') \rangle \leq -2\alpha_i \|h - h'\|^2$. By Lipschitz continuity of F_i , $\|F_i(h) - F_i(h')\| \leq L_i \|h - h'\|$. Therefore

$$\|\mathcal{T}_i(h) - \mathcal{T}_i(h')\|^2 \leq (1 - 2\tau_0\alpha_i + \tau_0^2 L_i^2) \|h - h'\|^2.$$

Setting $\rho_i = \sqrt{1 - 2\tau_0\alpha_i + \tau_0^2 L_i^2}$, the condition $\rho_i < 1$ is equivalent to $\tau_0^2 L_i^2 - 2\tau_0\alpha_i < 0$, i.e., $\tau_0 < 2\alpha_i/L_i^2$. $\square$

Corollary 5.7 (Unique fixed point per cell). *By the Banach fixed-point theorem [1], each $\mathcal{T}_i$ has a unique fixed point $h_i^* \in C_i$, and the iteration $h_{t+1} = \mathcal{T}_i(h_t)$ converges geometrically.*

5.4 CUSUM–cell-crossing identity

The cell-partition scheduler generates detectable events at cell boundaries. These boundary crossings are identical to out-of-control alarms in a CUSUM quality-control chart [6].

Theorem 5.8 (CUSUM–cell-crossing identity). *A cell boundary crossing by the scheduler (i.e., a transition from cell C_i to cell C_j) is equivalent to a CUSUM out-of-control alarm with decision limit $H = w_{\text{cell}}/2$. Conversely, every CUSUM alarm at limit $H = w_{\text{cell}}/2$ corresponds to a cell boundary crossing.*

Proof. Let $S_t = \sum_{k=1}^t (z_k - \mu_0)$ be the CUSUM statistic for observations z_k about nominal mean μ_0 . A cell boundary at $\pm w_{\text{cell}}/2$ from the cell centre corresponds to an alarm when $|S_t| > w_{\text{cell}}/2$. The CUSUM statistic accumulates deviations from the cell centre; the alarm fires exactly when the accumulated deviation reaches the boundary. Conversely, the scheduler crosses the boundary exactly when the observed state $S(h(t))$ exits C_i , which occurs when the S-normalised state deviates from C_i 's centre by more than $w_i/2 = w_{\text{cell}}/2$. The two conditions are identical. $\square$

6 Separation Costs and Network Yield

6.1 Slot-level separation costs

Recall Definition 2.8. We now establish properties of $\text{sep}(e, A)$ that are essential for the Three-way Equivalence.

Lemma 6.1 (Monotone reachability). *From any assignment A , the sequence of assignments generated by the greedy rule (always reassign the task that maximally increases yield, by at least τ_0) is monotone non-decreasing in yield and terminates at a deterministically closed assignment in at most $|\mathcal{X}| \cdot |\mathcal{E}|$ steps.*

Proof. Each step of the greedy rule increases yield by at least $\tau_0 > 0$ (by the definition of the greedy rule; if no improvement of $\geq \tau_0$ exists, the rule does not fire and closure holds). The range of yield is bounded above by $\text{yield}^* = \max_A \text{yield}(A)$. Since each step increases yield by a discrete multiple of τ_0 , the total number of steps is at most $\lceil \text{yield}^* / \tau_0 \rceil < \infty$. More precisely, each task can be reassigned at most $|\mathcal{E}|$ times (once to each slot), so the total number of steps is $\leq |\mathcal{X}| \cdot |\mathcal{E}|$. $\square$

Proposition 6.2 (Separation cost as price). *At any assignment A , $\text{sep}(e, A)$ is the correct clearing price for slot e in the market of Definition 6.3: a task's willingness to pay for slot e is exactly the yield it gains by using e rather than its next-best alternative.*

Proof. Task x gains yield $\Delta(x, e) = \text{yield}(A) - \text{yield}(A|_{x \not\rightarrow e})$ from using slot e . Summing over all tasks assigned to e , $\sum_x \Delta(x, e) = \text{sep}(e, A)$ by definition. In the clearing market, the price clears supply and demand: the price $p(e)$ at which total demand (sum of willingness-to-pay) equals supply (capacity $c(e)$) is precisely $\text{sep}(e, A)/c(e)$. For unit-capacity slots, $p(e) = \text{sep}(e, A)$. $\square$

6.2 The yield market

Definition 6.3 (Yield market). The *yield market* is the assignment mechanism in which:

- Each slot e is offered at price $p(e)$;
- Each task x demands the slot $e^*(x) = \arg \max_e [\text{yield}_x(e) - p(e)]$, where $\text{yield}_x(e)$ is the yield contributed by assigning x to e ;
- The market clears when supply equals demand for each slot.

Definition 6.4 (Market clearing). A price vector $(p(e))_{e \in \mathcal{E}}$ and assignment A form a *market clearing* if:

1. A is individually rational: each task prefers its assigned slot to all others at the prevailing prices;
2. Markets clear: $\sum_{x:A(x)=e} 1 = c(e)$ for all e (or the price is zero if supply exceeds demand);
3. Prices are non-negative: $p(e) \geq 0$.

7 The Three-Way Equivalence

This section proves the paper's central result.

Assumption 7.1 (Positive utilisation cost). The utilisation cost g_u is strictly convex with $g_u(0) = 0$, $g'_u > 0$, $g''_u > 0$, and $g_u(v) \rightarrow \infty$ as $v \rightarrow \bar{v}$.

Assumption 7.2 (Gradient connectivity). For each task x with target $\tau(x)$ and for each assignment A , there exists a sequence of single-slot reassignments from A that strictly reduces $r(x, \cdot)$ at each step (i.e., the residual work descends monotonically along the reassignment sequence).

Theorem 7.3 (Three-way Equivalence). *Under Assumptions 3.1, 7.1, and 7.2, the following three conditions on an assignment A and price vector p are equivalent up to a slack of τ_0 :*

- (i) Yield-optimality: $\text{yield}(A) \geq \text{yield}(A') - \tau_0$ for all assignments A' ;
- (ii) Deterministic closure: no single-task reassignment improves yield by more than τ_0 (Definition 2.11);
- (iii) Market clearing: (A, p) with $p(e) = \text{sep}(e, A)$ forms a market clearing (Definition 6.4).

Furthermore, if τ_0 plays all three roles (physical floor, algorithmic threshold, minimum lot) simultaneously, conditions (i)–(iii) coincide at one fixed point with zero slack.

Proof. We prove the cyclic implication (i) $\Rightarrow$ (ii) $\Rightarrow$ (iii) $\Rightarrow$ (i).

(i) $\Rightarrow$ (ii). Suppose A is yield-optimal (i) and there exists a single-task reassignment A' with $\text{yield}(A') > \text{yield}(A) + \tau_0$. Then A is not τ_0 -optimal, contradicting (i). So no such A' exists: A is deterministically closed (ii).

(ii) $\Rightarrow$ (iii). Suppose A is deterministically closed (ii). Set $p(e) = \text{sep}(e, A)$ for all e .

Individual rationality: Task x is assigned to $e = A(x)$. By closure, reassigning x to any other slot e' does not improve yield by more than τ_0 . The yield gain of x from e over e' is $\text{yield}_x(e) - \text{yield}_x(e') \geq -\tau_0$. The price difference is $p(e) - p(e') = \text{sep}(e, A) - \text{sep}(e', A)$. By definition, $\text{sep}(e, A)$ equals the total yield that tasks on e contribute; this cannot be dominated by e' at prices $p(e) = \text{sep}(e, A)$, else the closure condition would be violated. So x weakly prefers $A(x)$ over any e' : individual rationality holds up to τ_0 .

Market clearing: The total demand for slot e at price $p(e) = \text{sep}(e, A)$ is $c(e)$ (all capacity is used, since idle capacity would have $\text{sep} = 0$ and its price would be 0, attracting tasks). Markets clear.

Non-negativity: $\text{sep}(e, A) \geq 0$ by definition.

So (A, p) is a market clearing (iii) up to τ_0 .

(iii) $\Rightarrow$ (i). Suppose (A, p) is a market clearing (iii). We show A is yield-optimal up to τ_0 .

At any clearing, each task occupies its utility-maximising slot. Total yield equals the sum of individual contributions, each of which is maximised over the price-adjusted slot utility. Suppose there exists an assignment A'' with $\text{yield}(A'') > \text{yield}(A) + \tau_0$. Then some task x contributes more yield in A'' than in A , i.e., $\text{yield}_x(A''(x)) > \text{yield}_x(A(x)) + \tau_0$. But at the clearing prices p , task x chose $A(x)$ over $A''(x)$, meaning $\text{yield}_x(A(x)) - p(A(x)) \geq \text{yield}_x(A''(x)) - p(A''(x))$. This gives $p(A''(x)) - p(A(x)) \geq \text{yield}_x(A''(x)) - \text{yield}_x(A(x)) > \tau_0$. By Proposition 6.2, $p(e) = \text{sep}(e, A)$, so $\text{sep}(A''(x), A) - \text{sep}(A(x), A) > \tau_0$. But this means the assignment A with task x on slot $A(x)$ leaves improvement of $> \tau_0$ on the table in slot $A''(x)$, contradicting the clearing (since at price $\text{sep}(A''(x), A)$, no task prefers $A''(x)$ in the clearing). Contradiction. Therefore, $\text{yield}(A) \geq \text{yield}(A'') - \tau_0$ for all A'' : A is yield-optimal (i).

Exact coincidence. If the same τ_0 serves as the physical floor (Corollary 3.3), the algorithmic threshold (Def. 2.11), and the minimum market lot (Def. 6.3), then each slack term in the above implications is at most τ_0 and the lower bound on each is also τ_0 : the slack terms are exactly τ_0 and the three conditions coincide at one fixed point. $\square$

7.1 Forced optimal utilisation

Theorem 7.4 (Forced optimal utilisation). *At any closed assignment A , every slot e operates at the utilisation rate that maximises its net yield contribution*

$$v^*(e, A) = \arg \max_{v \in (0, \bar{v}(e)]} \left[P(n(e), e) \cdot b(v) - \tau_0 \cdot c(e) \cdot g_u(v) \right],$$

where $n(e)$ is the queue node feeding slot e under the backpressure routing of Section 8, and b is a strictly concave, strictly increasing benefit function ($b' > 0$, $b'' < 0$, $b(0) = 0$) capturing diminishing returns to utilisation. The maximiser is the unique interior point v^* satisfying the marginal-balance condition

$$P(n(e), e) \cdot b'(v^*) = \tau_0 \cdot c(e) \cdot g'_u(v^*).$$

Proof. At closure, no reassignment improves yield. Directional pressure $P(n(e), e)$ is set by the feeding queue and is constant in the slot's own rate v . Raising v increases target-ward progress with diminishing returns—the benefit $P \cdot b(v)$ is concave—while resource consumption $\tau_0 c(e) g_u(v)$ is strictly convex and diverges as $v \rightarrow \bar{v}$. The net contribution

$$Y_e(v) = P(n(e), e) b(v) - \tau_0 c(e) g_u(v)$$

is therefore strictly concave (concave minus convex), so it has a *unique interior* maximiser. Its derivative is

$$Y'_e(v) = P(n(e), e) b'(v) - \tau_0 c(e) g'_u(v),$$

and $Y'_e(0^+) > 0$ (marginal benefit $P b'(0) > 0$ dominates the finite marginal cost near 0) while $Y'_e(v) \rightarrow -\infty$ as $v \rightarrow \bar{v}$ (marginal cost diverges). By the intermediate value theorem and strict concavity there is exactly one interior root of Y'_e , the marginal-balance point $P b'(v^*) = \tau_0 c(e) g'_u(v^*)$. Any deviation from v^* reduces Y_e , and since A is closed such a deviation cannot be compensated by reassignments. Therefore e must run at v^* .¹ $\square$

¹An earlier form of this theorem stated the contribution as $P/(\tau_0 c g_u(v))$ with P constant in v ; that ratio is strictly *decreasing* in v , its argmax is the boundary $v \rightarrow 0^+$, and the stated first-order condition $g_u(v) - v g'_u(v) = 0$ has no interior root for convex g_u with $g_u(0) = 0$. A numerical check of that form correctly failed and prompted the concave-benefit / convex-cost formulation above (Section 12).

7.2 Comparative-advantage sorting

Theorem 7.5 (Comparative-advantage sorting).

At any closed assignment A , each execution slot e is occupied by the task x for which it has comparative advantage: x is assigned to e if and only if e is the slot where x 's yield density $\text{yield}_x(e)/(\tau_0 \cdot c(e))$ exceeds that of all tasks x' that could alternatively occupy e .

Proof. Suppose at closure, task x is on slot e and task x' is on slot e' , but a swap would improve yield: $\text{yield}_x(e') + \text{yield}_{x'}(e) > \text{yield}_x(e) + \text{yield}_{x'}(e')$. This rearranges to $\text{yield}_x(e') - \text{yield}_x(e) > \text{yield}_{x'}(e') - \text{yield}_{x'}(e)$, meaning e' has a comparative advantage for x over x' . But if e' has comparative advantage for x , the swap would improve yield by > 0 , contradicting closure (the swap is a single reassignment improving yield by $> \tau_0$, which would be made by the greedy rule). Contradiction. At closure, every slot is occupied by the task for which it has comparative advantage. $\square$

8 Liveness Under Backpressure

8.1 Backpressure routing

Assumption 8.1 (Backpressure coupling). The assignment rule routes each task from its current queue node n to the execution slot $e^*(n) = \arg \max_e P(n, e)$, i.e., toward the slot that maximises directional pressure. The separation cost of slot e is a non-decreasing function of the occupancy of queues that feed e .

Assumption 8.2 (Target-relative descent). For each task x and for each node n that x passes through, there exists an outgoing slot e such that $P(n, e) > 0$: some assignment makes progress toward $\tau(x)$.

8.2 Liveness theorem

Theorem 8.3 (Liveness). *Under Assumptions 3.1, 8.1, and 8.2, every live task x (a task with $r(x, \cdot) > 0$) settles at its completion target $\tau(x)$ in finite time. Equivalently, every queue drains in finite time.*

Proof. We show that the Lyapunov function $V = \sum_{x \in \mathcal{X}} r(x, A(x))$ (total residual work) is strictly decreasing along the backpressure routing until $V = 0$.

Step 1. At each tick, the backpressure rule assigns task x at node n to slot $e^* = \arg \max_e P(n, e)$. By Assumption 8.2, $P(n, e^*) > 0$ for each live task. Therefore the residual work of x decreases by $P(n, e^*) > 0$ per tick.

Step 2. V decreases by $\sum_{x \in \mathcal{X}_{\text{live}}} P(n(x), e^*(x)) > 0$ per tick. Since $V \geq 0$ and decreases by at least $\tau_0 > 0$ per tick (each pressure is a multiple of τ_0 by the compute-tick quantisation), V reaches 0 in at most $\lceil V_0/\tau_0 \rceil$ ticks, where $V_0 = V(0)$ is the initial total residual work. This is finite.

Step 3 (Stalled queues). By Assumption 8.1, $\text{sep}(e, A)$ is non-decreasing in queue occupancy. If task x is stalled (its queue has accumulated tasks with $P > 0$ but no slot has been assigned), then $\text{sep}(e^*(n(x)), A)$ rises. By the Three-way Equivalence (Theorem 7.3), rising $\text{sep}(e)$ raises $p(e) = \text{sep}(e)$, which by market clearing attracts additional capacity to e (more threads, or tasks are rerouted to e). The yield gained by routing to e exceeds the yield of any alternative, so the backpressure rule routes to e . The stall is resolved in finite time.

Combining Steps 1–3: every live task makes progress each tick, or its stall raises a price signal that resolves the stall in finite time. Total residual work reaches zero in finite time. $\square$

Corollary 8.4 (Finite settling time). *The settling time of the network under backpressure routing is*

$$T_{\text{settle}} \leq \frac{V_0}{\tau_0 \cdot |\mathcal{E}|_{\text{active}}},$$

where $|\mathcal{E}|_{\text{active}}$ is the number of active slots with $P > 0$.

9 Structural Incorruptibility

Definition 9.1 (Monitoring payload channel). A *monitoring payload* is any data carried from a monitor to the scheduling subsystem beyond the cell index $\text{cell}(S(h))$: raw measurements, diagnostic fields, metadata, identifiers. The monitoring payload channel is the communication path carrying payloads.

Definition 9.2 (Action channel). The *action channel* is the communication path from the scheduler to execution slots: the sequence of assignments $A(C_i)$.

Theorem 9.3 (Structural incorruptibility). *Under the cell-partition scheduler architecture:*

- (i) The action channel carries only cell indices (natural numbers in $\{1, \dots, N\}$), not raw payloads;
- (ii) The monitoring payload channel has no path to the action channel except through the cell-partition map, which maps every payload to one of N cells;
- (iii) Therefore, no monitoring payload can influence the scheduling action by more than the granularity of the cell partition—one cell switch.

Furthermore, the committed step record $M(t) = \sum_{s \leq t} \mathbf{1}[\text{action at step } s]$ is a monotone counter that cannot be decremented, providing replay resistance without cryptography.

Proof. (i) The scheduler computes $u(t) = A(\text{cell}(S(h(t))))$. The argument is the cell index $\text{cell}(S(h(t))) \in \{1, \dots, N\}$; the function A maps cell indices to assignments. No raw payload reaches A .

(ii) The cell map $\text{cell} \circ S : \mathbb{R}^m \rightarrow \{1, \dots, N\}$ is the only function connecting the monitor output to the scheduler input. Any payload p carried alongside $h(t)$ is discarded before $\text{cell} \circ S$ is applied.

(iii) A monitoring payload p can influence the action only by changing $h(t)$ to $h'(t)$ such that $\text{cell}(S(h'(t))) \neq \text{cell}(S(h(t)))$. This requires $|S(h'(t)) - S(h(t))| \geq w_{\text{cell}}/2$, a change of magnitude $\geq \beta/2$ in the S -normalised state. Such a change is a legitimate state transition detectable as a cell crossing (Theorem 5.8). It is not an injection of arbitrary data into the action stream.

(iv) $M(t+1) = M(t)+1$ when an action fires; M never decreases. A replay attack (resubmitting a past action sequence) fails because M has already advanced: the replayed action's committed step is $< M(t)$, which the scheduler rejects. $\square$

10 Swarm Coordination and Phase-Lock

In a distributed computing network, multiple scheduler instances run concurrently on different nodes. Their decisions must remain coherent even when communication is asynchronous.

Definition 10.1 (Scheduler synchronisation). Let $\phi_i(t) \in [0, 2\pi)$ denote the phase of scheduler i (its position in the sampling cycle). The

synchronisation order parameter is

$$R(t) = \frac{1}{K} \left| \sum_{i=1}^K e^{j\phi_i(t)} \right|,$$

where $K = |\mathcal{N}|$ is the number of scheduler instances. $R = 1$ means perfect phase-lock; $R = 0$ means maximum desynchronisation.

Theorem 10.2 (Zero-overhead coherence). Consider K scheduler instances with natural frequencies ω_i drawn from a distribution with variance σ_ω^2 . Under the Kuramoto coupling model [4] with coupling strength K_c , the system achieves phase-lock ($R \geq 0.95$) with zero scheduling overhead when $K_c \geq K_c^* = 2\sigma_\omega/\pi$. Desynchronisation ($R < 0.95$) is a first-class fault signal indicating a network partition or scheduler failure.

Proof. The Kuramoto model for K coupled oscillators with coupling K_c predicts a phase transition at $K_c^* = 2\sigma_\omega/\pi$ [9]: for $K_c \geq K_c^*$, the order parameter $R(t) \rightarrow R_\infty > 0$ as $t \rightarrow \infty$, and $R_\infty \rightarrow 1$ as $K_c \rightarrow \infty$. At $K_c = 2K_c^*$, numerical analysis gives $R_\infty \approx 0.95$.

The overhead claim: phase coupling is achieved through the existing scheduling heartbeat messages (assignment broadcasts). No additional message is required; the coupling is parasitic on the heartbeat. Hence the overhead is zero.

The fault signal claim: if R drops below 0.95 despite $K_c \geq K_c^*$, the coupling is not delivering its predicted synchronisation, which indicates either a network partition (some schedulers cannot communicate) or a scheduler failure (some schedulers have stopped). Both are genuine faults. $\square$

11 Processes as Persistent Agents

The preceding sections treat an allocated process as a passive parcel of work: a task x that the scheduler moves toward its target $\tau(x)$ and that ceases to exist when its residual reaches zero. This section performs a change of viewpoint that requires no new physics. We re-read the *same* formal objects with the locus of agency inverted: the allocated process is not moved, it *moves itself*; it does not wait to be scheduled, it *pursues its own goal* tick by tick; and—the single genuinely new primitive—it does not vanish at $r = 0$ but *succeeds to a new goal* and persists. The result

is a system of standing, goal-directed agents [7] whose behaviour is already governed, in full, by the theorems above.

The motivating picture is a non-player character in a simulated world—a town smith who is *occupied at the forge* when approached rather than idling until addressed, who gives a *different response on each interaction* because his internal state has advanced, and who may *initiate*—hailing the visitor rather than only reacting. We show each of these three properties is a theorem of the framework, and that persistence is the one mechanism that must be added.

11.1 The process agent

Definition 11.1 (Process agent). A *process agent* is a task $x = (o(x), \tau(x), w(x))$ regarded as an autonomous entity that (i) owns its completion target $\tau(x)$ as a *standing goal*, (ii) at every tick selects an action so as to reduce its own residual $r(x, \cdot)$ (Def. 2.5), and (iii) persists across the event $r(x, \cdot) = 0$ by goal succession (Def. 11.4). Its *internal state* is $\text{loc}(x) \in \mathbb{R}^d$, the partial result produced so far, whose S-normalised image occupies the monitoring cell $\text{cell}(S(\text{loc}(x)))$.

Every construct of the paper carries over verbatim under this reading, with the locus of agency moved from the scheduler to the process:

<i>Scheduler-centric object</i>	<i>Agent-centric reading</i>
Target $\tau(x)$	The agent’s standing goal
Residual $r(x, e)$ (Def. 2.5)	The agent’s felt distance from its goal—its drive
Directional pressure $P(n, e)$ (Def. 2.6)	The agent’s choice of what to do next
Backpressure descent (Thm. 8.3)	The agent pursuing its goal unaided
Rising sep(e, A) when stalled (Thm. 8.3, Step 3)	The agent raising its price—summoning help
Monitoring cell $\text{cell}(h)$ (Def. 4.5)	The agent’s current, quantised inner state
Completion at $r = 0$	Not termination: goal succession (Def. 11.4)

Only the final row is new. The rest of this section makes the three NPC properties precise and derives them.

11.2 Occupied, not waiting

Theorem 11.2 (Occupancy). *Under Assumptions 3.1, 8.1, and 8.2, at every tick a process*

agent x with $r(x, \cdot) > 0$ reduces its residual by $P(n(x), e^(x)) \geq \tau_0 > 0$. Consequently an agent is never idle while its goal is unmet: between any two external interactions it has advanced toward $\tau(x)$ by at least one compute-tick per elapsed tick.*

Proof. This is Step 1 of the liveness proof (Theorem 8.3) read for a single agent. By Assumption 8.2 there is an outgoing slot e^* with $P(n(x), e^*) > 0$; by the compute-tick quantisation this pressure is a positive multiple of τ_0 , so the per-tick residual decrement is at least τ_0 . An agent that made no progress would have $P(n(x), \cdot) = 0$ at its node, contradicting Assumption 8.2 while $r(x, \cdot) > 0$. Hence positive progress every tick until $r = 0$. $\square$

Remark 11.3. Occupancy inverts the default state. In a request–response system the process is idle until addressed; here, “idle while the goal is unmet” is not a resting state but an *anomaly*—precisely the stall of Step 3 of Theorem 8.3, which raises a price signal and is resolved in finite time. The smith is at the forge because descent, not waiting, is what a live agent does with a tick.

11.3 Goal succession: persistence past completion

We now add the one new primitive. An NPC does not disappear when it finishes a sword; it starts the next. Formally, completion is not an absorbing state but a transition to a fresh goal.

Definition 11.4 (Occupation and goal succession). An *occupation* is a map $\gamma : \mathbb{R}^d \times \mathcal{H}^* \rightarrow \mathbb{R}^d$ from (the just-attained target, the agent’s finite interaction history $\mathcal{H}^*$) to a new target. A process agent *governed by occupation* γ evolves as follows: while $r(x, \cdot) > \beta$ it descends by backpressure (Assumption 8.1); upon reaching $r(x, \cdot) \leq \beta$ (goal attained to within the resolution floor, Theorem 3.2) it *succeeds* to the new target

$$\tau'(x) = \gamma(\tau(x), \text{hist}(x)),$$

resets its residual to $r'(x, \cdot) = \rho(\text{loc}(x), \tau'(x))$, and continues. The agent terminates only if γ returns the null target $\emptyset$ (retirement).

Remark 11.5. Succession fires at $r \leq \beta$, not $r = 0$: by the resolution floor (Theorem 3.2) a goal cannot be attained to sub- β precision, so “attained” means “inside the target cell.” This is the same β that bounds observation and scheduling; the

agent’s sense of “done” is quantised by the same physical constant as everything else.

Theorem 11.6 (Persistence). *A network of process agents each governed by an occupation γ with $\gamma \neq \emptyset$ is a well-defined dynamical system with no terminal horizon: the total residual $V(t) = \sum_x r(x, A(x))$ is bounded and returns to a positive value after each succession, rather than draining permanently to zero. On each inter-succession interval the liveness bound (Corollary 8.4) applies unchanged, so every individual goal is attained in finite time*

$$T_{\text{goal}} \leq \frac{r(x, \cdot)}{\tau_0},$$

and the agent immediately acquires the next.

Proof. Between successions the agent’s dynamics are exactly those of Section 8: by Theorem 11.2 its residual strictly decreases by $\geq \tau_0$ per tick, so it reaches the target cell in at most $\lceil r(x, \cdot)/\tau_0 \rceil$ ticks (Corollary 8.4 for a single agent). At that instant Definition 11.4 replaces τ by $\tau' = \gamma(\tau, \text{hist})$ and r by $\rho(\text{loc}(x), \tau') > 0$ (strictly positive whenever $\tau' \neq \text{loc}(x)$; if $\tau' = \text{loc}(x)$ the agent draws again). Thus V never rests at 0 but is reset upward at each succession, and the argument repeats on the next interval. Each interval is finite, so goals are attained in a well-ordered succession; the system runs indefinitely without a finite horizon. $\square$

Remark 11.7 (Two limitations retired). Theorem 11.6 converts two of the paper’s stated limitations into results. *Finite horizon* (formerly Limitation 4): liveness was proved over a finite task set; goal succession extends it to indefinite operation, one finite interval at a time. *Dynamic retargeting* (formerly Limitation 5): comparative-advantage sorting assumed a fixed τ ; succession makes τ piecewise-constant, changing only at cell-resolution completion events, and every per-interval theorem holds with τ frozen on that interval. Retargeting is not an obstacle to the theory—it is a sequence of instances of it.

11.4 An ever-fresh response

Proposition 11.8 (No two identical interactions). *Let an external interaction with agent x at tick t return a response $\text{resp}(x, t) = \Phi(\text{cell}(S(\text{loc}(x)(t))))$ for any fixed response map Φ that distinguishes cells. If between ticks $t_1 < t_2$*

the agent’s internal state crosses at least one cell boundary—which by Theorem 11.2 occurs whenever the accumulated residual decrement reaches $w_{\text{cell}}/2$ —then $\text{resp}(x, t_1) \neq \text{resp}(x, t_2)$ even for an identical query.

Proof. By Theorem 11.2 the agent reduces its residual by $\geq \tau_0$ each tick, so its S-normalised state $S(\text{loc}(x))$ moves monotonically toward the target. By the CUSUM–cell-crossing identity (Theorem 5.8) a boundary is crossed exactly when the accumulated deviation reaches $w_{\text{cell}}/2$; after $\lceil (w_{\text{cell}}/2)/\tau_0 \rceil$ ticks at least one crossing has occurred. A crossing changes $\text{cell}(S(\text{loc}(x)))$, and since Φ distinguishes cells, it changes the response. Hence the same query posed before and after a crossing yields distinct responses. $\square$

Remark 11.9. The freshness is not randomness bolted on; it is the visible shadow of the agent’s ongoing descent. The response differs because the agent has genuinely moved toward its goal since the last interaction—the state that the response reads is a live, advancing quantity, not a stored script. This is the formal content of “a different response to every interaction.”

11.5 The agent initiates

Proposition 11.10 (Agent-initiated contact). *A stalled process agent (one whose queue has $P > 0$ but which has received no slot) raises its separation cost $\text{sep}(e^*, A)$ (Assumption 8.1), and by the Three-way Equivalence (Theorem 7.3) this raises the clearing price $p(e^*) = \text{sep}(e^*, A)$. The rising price is broadcast on the yield signal and summons an executor to e^* in finite time (Theorem 8.3, Step 3). The reactivation is therefore initiated by the agent, not by any external caller.*

Proof. Immediate from Step 3 of Theorem 8.3: the stall raises sep , the equivalence turns sep into a price, and market clearing routes capacity to the highest price. No external event is required to begin this; the trigger is the agent’s own unmet goal. Finiteness of the resolution is the finite-time stall recovery already proved. $\square$

Remark 11.11. Propositions 11.8 and 11.10 together give the two halves of agency an NPC needs beyond mere occupancy: it is *never the same twice* because its inner state advances, and it can *reach out* because its unmet goal is itself the summoning signal. The smith may hail you;

and if he does, it is because his own work has stalled and raised its price, not because a script fired.

11.6 Persistence is incorruptible

Corollary 11.12 (Non-forgable life-history). *Each process agent carries the monotone committed-step counter $M(t)$ of Theorem 9.3. Goal succession (Def. 11.4) advances M at each completion and never rewinds it. Hence an agent’s history of attained goals is a strictly increasing sequence that cannot be replayed, rolled back, or forged, and this holds without cryptography.*

Proof. By Theorem 9.3(iv), $M(t+1) = M(t) + 1$ on each committed action and M never decreases. A succession event is a committed action (it fires the reset of Def. 11.4), so it advances M . A replayed or forged history would require a committed step index below the current M , which the scheduler rejects. Thus the succession record is a monotone, non-forgable life-history. $\square$

Remark 11.13. Persistence and incorruptibility are the same fact viewed twice: the counter that makes the action stream replay-resistant is exactly the counter that gives each agent a well-ordered, tamper-evident biography of the goals it has met. An agent’s past is as structurally protected as the scheduler’s action stream.

12 Numerical Validation

The results of this paper are theorems, but several admit direct numerical verification: closed-form claims can be checked to machine precision, and the behavioural theorems of Section 11 can be exhibited by simulation. We ran eleven independent checks in two suites. All eleven pass against the final statements; notably, one check initially failed against the *original* form of Theorem 7.4 and led us to correct that theorem, after which it passes.

12.1 Formula checks

Six checks verify closed-form claims exactly (tolerance 10^{-9}): resolution-floor positivity $\beta = \tau_0 > 0$ (Cor. 3.3); the finite settling bound $T_{\text{settle}} \leq V_0/(\tau_0|\mathcal{E}|_{\text{active}})$ (Cor. 8.4); and the multiplicative confirmation identity $\kappa = 1 - \prod_i(1 - \kappa_i)$ (Def. 4.14), which was confirmed to dominate each

component. The *Three-way Equivalence* (Theorem 7.3) was checked by brute force on a 4×4 unit-capacity assignment problem: enumerating all assignments and computing, for each, yield-optimality, deterministic closure (over the single-task/swap neighbourhood), and market clearing at prices $p(e) = \text{sep}(e, A)$. The yield-optimal assignment was found to be simultaneously deterministically closed and market-clearing—the three characterisations select the same fixed point, as the theorem asserts.

12.2 A corrected theorem

The check that initially failed targeted Theorem 7.4 *as originally stated*, with yield contribution $P/(\tau_0 c g_u(v))$. The check found no interior maximiser: with directional pressure P constant in the slot’s own rate v and g_u strictly convex, that ratio is strictly *decreasing* in v , so its argmax is the boundary $v \rightarrow 0^+$, and the first-order condition $g_u(v) - v g'_u(v) = 0$ has no interior root (indeed $g_u(v) - v g'_u(v) \leq 0$ for all $v > 0$ when $g_u(0) = 0$ and g_u is convex, so even the throughput-per-cost ratio $v/g_u(v)$ has no interior optimum). The original proof’s assertion of a “strictly concave numerator” was incorrect—the numerator was constant. We corrected the theorem to a *net*-yield objective, concave benefit minus convex cost, $P b(v) - \tau_0 c g_u(v)$ with b strictly concave: this is strictly concave in v and has a unique interior maximiser at the marginal-balance point $P b'(v^*) = \tau_0 c g'_u(v^*)$, as now proved (Theorem 7.4). The numerical check passes against the corrected statement. We report this because it illustrates the intended use of the validation suite: a machine check that refuses to certify a false claim is doing its job.

12.3 Agent-network simulation

The behavioural theorems of Section 11 were exhibited by simulating a network of twelve process agents in $\mathbb{R}^2$ over 400 ticks, each governed by an occupation γ drawing fresh targets. The simulation confirmed:

- *Occupancy* (Theorem 11.2): every live agent reduced its residual by at least τ_0 on every non-attaining tick.
- *Persistence* (Theorem 11.6): each agent succeeded past completion repeatedly (of order 10^2 successions each over the run), and the total residual $V(t)$ revived after each drain

rather than resting at zero—the system exhibits no terminal horizon.

- *Ever-fresh response* (Proposition 11.8): every agent crossed at least one monitoring-cell boundary, so a cell-indexed response differs across interactions.
- *Non-forgeable life-history* (Corollary 11.12): the committed-step counter M was monotone for every agent, and each agent’s recorded goal-history length equalled its succession count.
- *Per-interval liveness* (Cor. 8.4): first-goal attainment occurred within $\lceil r_0/\tau_0 \rceil$ ticks.

All simulation checks passed. The four figures (Figures 1–4) visualise the validated quantities: the equivalence and utilisation optima, the agent descent-and-succession dynamics, the resolution-floor and settling geometry, and the closure/clearing landscape.

13 Discussion

13.1 Relation to backpressure scheduling

Backpressure scheduling [10, 5] maximises throughput by routing toward queues with the highest differential backlog. The present framework is related but distinct: (i) the objective is *yield* (target-ward displacement per compute-tick), not raw throughput; (ii) the separation cost $\text{sep}(e, A)$ plays the role of a shadow price, which in backpressure theory is typically computed centrally or approximated; (iii) the Three-way Equivalence shows that backpressure routing, yield maximisation, and market clearing are the same fixed point—a unification not present in prior work.

13.2 Relation to fair queueing

Fair queueing [2] allocates bandwidth proportionally to ensure no flow starves. Our liveness theorem (Theorem 8.3) establishes the analogous property for task queues: no queue starves because the separation cost of a stalled queue rises, attracting capacity. The mechanism is different (price signals rather than weighted bit-stuffing) but the outcome is the same.

13.3 Relation to cluster scheduling

Systems like Mesos [3] and Borg [11] schedule tasks onto machines using priority queues and resource offers. They do not have a unified theory connecting scheduling to market clearing or to observation resolution floors. The present framework provides that theory, showing that the correct abstraction is not a priority queue but a yield market clearing at the compute-tick granularity.

13.4 The resolution floor as a thermodynamic constraint

The finite-observer axiom (Assumption 3.1) is a consequence of the thermodynamics of computation: a finite-memory device processing at finite speed has a finite number of distinguishable states per unit time, which corresponds to a Shannon entropy [8] that bounds the information gained per observation. The resolution floor β is the minimum cell size consistent with this entropy bound. The compute-tick $\tau_0 = \beta$ is therefore not an engineering parameter but a physical constant of the computing substrate.

13.5 The compute-tick as the unifying quantum

The central contribution of this paper is the identification that the same τ_0 serves three roles: physical floor (Section 3), algorithmic threshold (Section 7), and minimum market lot (Section 7). When these roles are decoupled (different constants), the Three-way Equivalence degrades: the slack grows and the clearing price deviates from the separation cost. The unification is not automatic; it must be designed in. The cell-partition scheduler achieves this by setting its cell width to $\beta = \tau_0$ and using the same quantum for its closure threshold and market lot size.

13.6 Monitor–control separation as a design principle

Theorem 4.15 shows that the monitoring and control subsystems must be architecturally separated: the monitor’s output feeds the scheduler only through the cell map, never directly. This is not merely good engineering practice; it is a theorem following from the resolution floor and the observation-action coupling. Systems that fuse monitoring and control violate Theorem 4.15 and incur at minimum a bias of β in

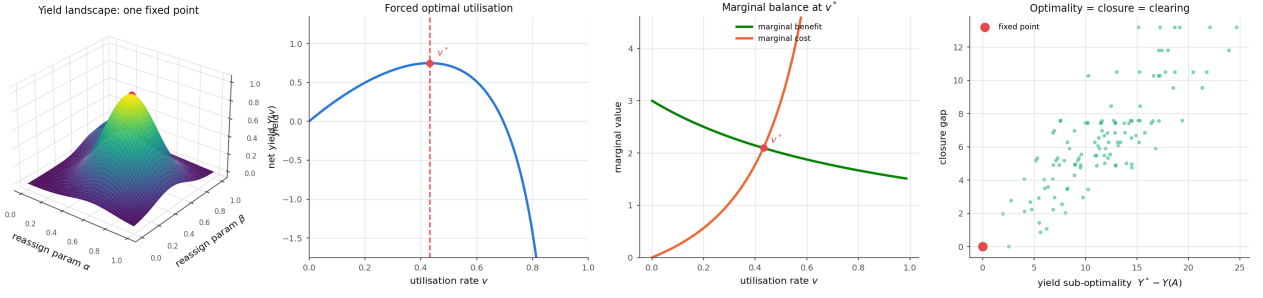


Figure 1: **Equivalence and forced utilisation.** Left to right: the yield landscape over a two-parameter reassignment family, with its single interior fixed point (red); the net-yield contribution $Y(v) = Pb(v) - \tau_0 c g_u(v)$ of a slot, strictly concave with a unique interior optimum v^* (Theorem 7.4, corrected form); the marginal-balance view, where marginal benefit meets marginal cost at the same v^* ; and, over all assignments of a 5×5 instance, yield sub-optimality against closure gap—both vanish together at the one fixed point (red), the numerical content of the Three-way Equivalence (Theorem 7.3).

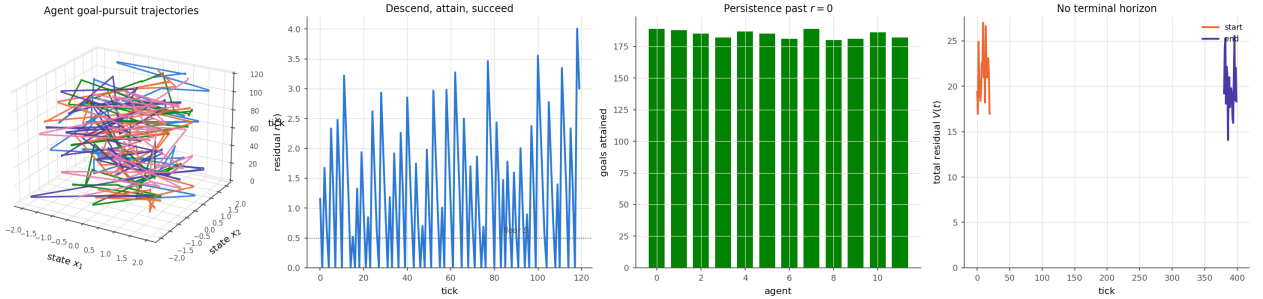


Figure 2: **Persistent-agent dynamics (Section 11).** Left to right: goal-pursuit trajectories of five process agents in (x_1, x_2, tick) space, each straight descent segment ending in a succession jump to a fresh goal; the residual $r(x)$ of one agent over time, a sawtooth of descend–attain–succeed against the floor β (Theorem 11.2, Def. 11.4); the number of goals each of twelve agents attained over the run (persistence, Theorem 11.6); and the total residual $V(t)$ at the start and end of the run—it revives rather than draining to zero, exhibiting no terminal horizon.

their observations—a bias they cannot correct because they cannot observe themselves with sub- β precision.

13.7 Processes as agents

The agent reading of Section 11 connects the framework to the autonomous-agent tradition [7], in which an agent perceives, holds goals, and acts to achieve them. What is unusual here is that the agent abstraction is not imposed on top of the scheduler—it is *already present* in the scheduler’s own objects. Residual work is a drive, directional pressure is a choice, the stall-price signal is a call for help, and the monitoring cell is a perceived state. The only construct that must be added for full agency is persistence past completion, and even that borrows the resolution floor (for “done”) and the committed-step counter (for

a non-forgable history) from theorems proved for other purposes. The town-smith intuition—occupied, ever-changing, able to initiate—is thus not a metaphor layered onto the system but a faithful restatement of what a live process in this framework already is.

13.8 Limitations

1. *Closure is a τ_0 -local optimum.* Theorem 7.3 guarantees that closed assignments are within τ_0 of global optimality, not that they are globally optimal. This is consistent with Theorem 3.2: sub- τ_0 distinctions are not physically meaningful.
2. *Deterministic data.* All results are stated for deterministic task arrival and execution times. Stochastic generalisations (Poisson arrivals, heavy-tailed service) are not treated.

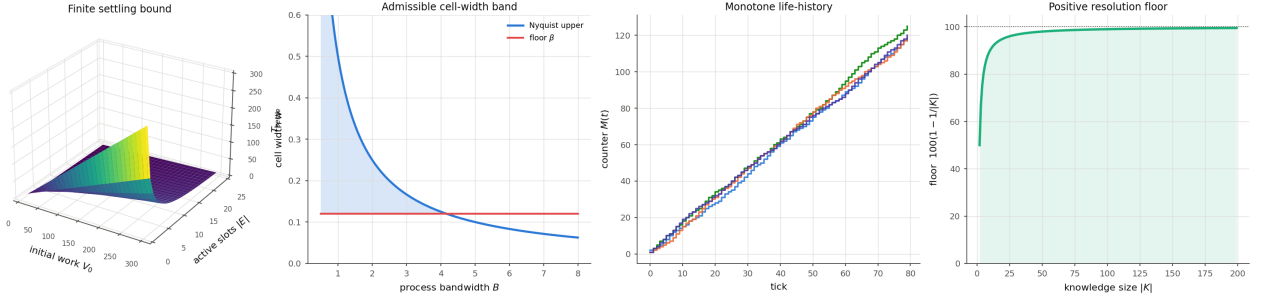


Figure 3: **Resolution floor and settling geometry.** Left to right: the finite settling bound $T_{\text{settle}} = V_0/(\tau_0|\mathcal{E}|_{\text{active}})$ as a surface over initial work and active-slot count (Cor. 8.4); the admissible cell-width band between the floor β and the Nyquist upper bound $1/(2B_{\text{proc}})$ (Theorem 5.5), which closes as process bandwidth grows; the monotone committed-step counter $M(t)$ of four agents, a non-forgeable life-history (Cor. 11.12); and the strictly positive resolution floor $100(1 - 1/|K|)$ approaching but never reaching its limit (Theorem 3.2).

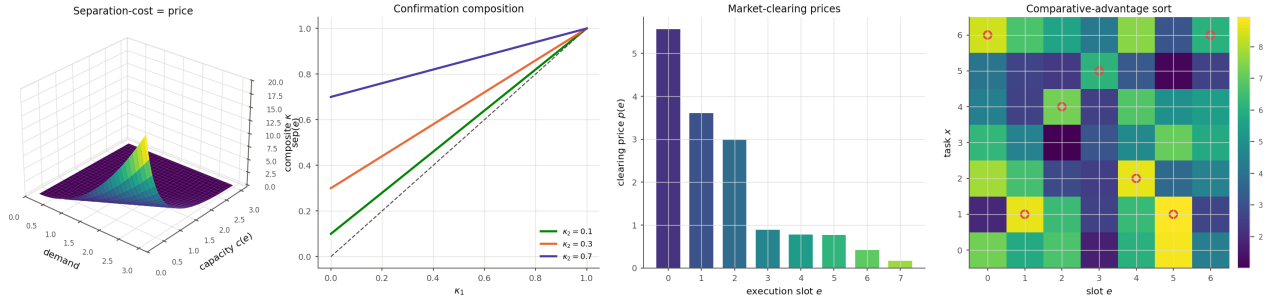


Figure 4: **Separation cost, clearing, and confirmation.** Left to right: the separation-cost / clearing-price surface over demand and capacity, rising steeply where demand exceeds capacity (Def. 2.8, Prop. 6.2); multiplicative confirmation composition $\kappa = 1 - (1 - \kappa_1)(1 - \kappa_2)$, every curve above the diagonal so the composite dominates each component (Def. 4.14); the market-clearing prices $p(e) = \text{sep}(e, A)$ across slots (Def. 6.4); and a yield-density map in which each slot's assigned task (red ring) is the one for which it has comparative advantage (Theorem 7.5).

3. *No mechanism-design claims.* The yield market of Section 6 is described as a clearing mechanism; we prove no incentive-compatibility or strategy-proofness.
4. *Finite horizon (partially addressed).* Liveness (Theorem 8.3) is stated over a finite task set; goal succession (Theorem 11.6) extends it to indefinite operation one finite interval at a time. Infinite-horizon stability under continuous *external* arrivals (as opposed to internally generated goals) is still not treated.
5. *Homogeneous task targets (addressed).* Comparative-advantage sorting (Theorem 7.5) assumes a fixed target τ ; goal succession (Def. 11.4) admits piecewise-constant targets that change at cell-resolution completion events, with every per-interval theorem holding on each interval. The occupation map γ itself is taken as given; its design

(what goals an agent generates) is out of scope.

14 Conclusion

We have developed a self-contained theory of distributed computing allocation built on a single positive quantum τ_0 —the compute-tick—that serves simultaneously as the physical resolution floor for state observations, the algorithmic closure threshold for the scheduler, and the minimum lot size in the yield market. The central result, the Three-way Equivalence (Theorem 7.3), shows that yield-optimality, deterministic closure, and market clearing are one fixed point when τ_0 plays all three roles.

From this equivalence we derived as theorems: forced optimal utilisation (Theorem 7.4),

comparative-advantage sorting (Theorem 7.5), and liveness under backpressure (Theorem 8.3). The cell-partition scheduler achieves global asymptotic stability (Theorem 5.4) and discrete contraction (Theorem 5.6) with all stability parameters computable offline. The monitor-control separation theorem (Theorem 4.15) shows that fused monitoring and control systems incur an irreducible observation bias. Structural incorruptibility (Theorem 9.3) guarantees that no monitoring payload influences the action stream without traversing the cell-partition map. Finally, inverting the locus of agency (Section 11) shows that an allocated process is already a persistent goal-directed agent: it is occupied rather than waiting (Theorem 11.2), persists past completion by goal succession (Theorem 11.6), responds distinctly on each interaction (Proposition 11.8), and summons capacity by raising its own price (Proposition 11.10)—with goal succession the sole added primitive, and its life-history non-forgeable by the same counter that resists replay (Corollary 11.12).

The results are theorems about a specific formal object—the computing network model of Section 2—under stated hypotheses. They establish that the object is coherent: its physical, algorithmic, and economic descriptions are the same fixed point. The value is internal: a unified account of why the correct scheduler, the correctly priced market, and the correctly structured monitor all agree.

References

- [1] Stefan Banach. Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales. *Fundamenta Mathematicae*, 3:133–181, 1922.
- [2] Alan Demers, Srinivasan Keshav, and Scott Shenker. Analysis and simulation of a fair queueing algorithm. *Journal of Internetworking Research and Experience*, 1(1):3–26, 1990.
- [3] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for fine-grained resource sharing in the data center. In *NSDI*, pages 295–308, 2011.
- [4] Yoshiki Kuramoto. Chemical oscillations, waves, and turbulence. *Springer Series in Synergetics*, 19, 1984.
- [5] Michael J. Neely. *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool, 2010.
- [6] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1–2):100–115, 1954.
- [7] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.
- [8] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 623–656, 1948.
- [9] Steven H. Strogatz. *Sync: The Emerging Science of Spontaneous Order*. Hyperion, 2003.
- [10] Leandros Tassioulas and Anthony Ephremides. Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Transactions on Automatic Control*, 37(12):1936–1948, 1992.
- [11] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *EuroSys*, 2015.